

Migration Report: CSV/JSON Storage to SQLite

Project: App-sensibilisation-video

November 20, 2025

Contents

1	Purpose	2
2	Baseline State (Pre-Migration)	2
3	Step 1: Environment Setup (Completed)	2
3.1	Objective	2
3.2	Actions	2
3.3	Result	2
4	Step 2: Database Initialization (Completed)	2
4.1	Objective	2
4.2	Artifacts Created	2
4.3	Schema Defined	2
4.4	Pragmas	3
4.5	Verification	3
5	Step 3: Data Migration (Completed)	3
5.1	Objective	3
5.2	Artifacts Created	4
5.3	What the script does	4
5.4	Field mapping	4
5.5	How we ran it	4
5.6	Verification	5
5.7	Idempotency and safety	5
6	Step 4: Replace Data Access Layer (Completed)	5
6.1	Objective	5
6.2	What changed	5
6.3	Developer notes	5
6.4	Implementation details	5
6.5	Data repair: normalize mis-parsed pairs (Completed)	7
7	Step 5: Adapt the Service Layer (Completed)	7
7.1	Objective	7
7.2	What changed	7
7.3	Implementation details	7
7.4	Verification	8
7.5	Notes	8
8	Change Log	8

1 Purpose

This living document records each step of migrating the project from ad-hoc CSV (`data.csv`) and JSON (`data.json`) storage to a structured SQLite database using the `better-sqlite3` Node.js library. It will be updated after every migration task (schema changes, data import, service refactors, etc.).

2 Baseline State (Pre-Migration)

- User session data appended to `data.csv` with 17 columns (user choices, resolutions, meta-data).
- Aggregated score/time data persisted in `data.json` (nested objects for scores, times, precisions).
- No database; all queries implied linear scans / in-memory aggregation.

3 Step 1: Environment Setup (Completed)

3.1 Objective

Add a performant, synchronous SQLite driver to project dependencies.

3.2 Actions

- 1.1 Installed dependency: `better-sqlite3@11.10.0`.
Added to `package.json` under `dependencies`.
- 1.2 Updated `package-lock.json` (36 packages added transitively).¹
- 1.3 No other scripts modified at this step.

3.3 Result

Project is now capable of creating and interacting with a local SQLite database file. No runtime code yet consumes the DB.

4 Step 2: Database Initialization (Completed)

4.1 Objective

Introduce a reproducible schema creation script and persistent database file.

4.2 Artifacts Created

- Directory: `db/`
- Script: `db/init-database.js`
- Database file (after execution): `db/database.db`
- Added npm script: `db:init: node db/init-database.js`

4.3 Schema Defined

Two tables established:

¹As reported by npm: “added 36 packages, audited 115 packages, 0 vulnerabilities”.

sessions Mirrors original CSV structure; adds surrogate primary key for internal references.

```
CREATE TABLE IF NOT EXISTS sessions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user TEXT NOT NULL,  
  category1 TEXT,  
  videoName1 TEXT,  
  videoPath1 TEXT,  
  resolution1 TEXT,  
  category2 TEXT,  
  videoName2 TEXT,  
  videoPath2 TEXT,  
  resolution2 TEXT,  
  QO1 TEXT,  
  QO2 TEXT,  
  QO3 TEXT,  
  QO4 TEXT,  
  QO5 TEXT,  
  comments TEXT,  
  screenType TEXT,  
  timestamp TEXT NOT NULL  
);  
CREATE INDEX IF NOT EXISTS idx_sessions_user ON sessions(user);  
CREATE INDEX IF NOT EXISTS idx_sessions_timestamp ON sessions(  
  timestamp);
```

users Stores aggregate metrics previously in JSON.

```
CREATE TABLE IF NOT EXISTS users (  
  pseudo TEXT PRIMARY KEY,  
  totalScore INTEGER DEFAULT 0,  
  totalTime INTEGER DEFAULT 0,  
  sessionCount INTEGER DEFAULT 0  
);
```

4.4 Pragmas

Enabled Write-Ahead Logging and foreign key enforcement:

```
PRAGMA journal_mode = WAL;  
PRAGMA foreign_keys = ON;
```

4.5 Verification

Execution output confirmed creation of tables: **sessions**, **users**, plus **sqlite_sequence** (auto-increment bookkeeping).

5 Step 3: Data Migration (Completed)

5.1 Objective

Preserve historical data by importing existing flat files into the new SQLite schema.

5.2 Artifacts Created

- Script: `db/migrate-data.js` (one-time migrator).
- Backups: `backups/data.csv.bak`, `backups/data.json.bak`.
- NPM command: `migrate:data` to run the migrator.

5.3 What the script does

- Reads `data.csv` and maps each row to the `sessions` table.
- Reads `data.json` and aggregates per-user totals into `users`.
- Wraps inserts in a single transaction; enables WAL and sets `synchronous=NORMAL` for faster batch writes.
- Supports a dry run via `DRY_RUN=1` to validate counts without writing.
- Uses an UPSERT for users so the script can be re-run safely without duplicating rows.

5.4 Field mapping

CSV (sessions) The following columns are mapped 1:1 from the CSV header to the table columns:

```
user , category1 , videoName1 , videoPath1 , resolution1 ,
category2 , videoName2 , videoPath2 , resolution2 ,
QO1, QO2, QO3, QO4, QO5,
comments , screenType , timestamp
```

If a field is missing in a row, it is inserted as NULL (except `user` and `timestamp`, which default to 'unknown' and current ISO time respectively).

JSON (users) The source file combines three objects: `scores`, `times`, and `precisions`. The script builds one record per pseudo as:

```
pseudo , totalScore <- scores[pseudo]
      totalTime  <- times[pseudo]
      sessionCount <- precisions[pseudo].sessions
```

Missing values default to 0. The users table is updated with an UPSERT so totals reflect the JSON content exactly.

5.5 How we ran it

Backups were created first and then the migration was executed:

```
mkdir -p backups
cp data.csv backups/data.csv.bak
cp data.json backups/data.json.bak
```

```
# Preview (no writes)
DRY_RUN=1 npm run migrate:data
```

```
# Execute migration
npm run migrate:data
```

5.6 Verification

After the run, we verified row counts using Node with `better-sqlite3`:

```
node -e "const Database=require('better-sqlite3');\nconst db=new Database('db/database.db');\nconsole.log('sessions:',db.prepare('SELECT COUNT(*) AS c FROM\n    sessions').get().c);\nconsole.log('users:',db.prepare('SELECT COUNT(*) AS c FROM users').\n    get().c);\n    db.close();"
```

Observed on this run: **116** sessions and **29** users.

5.7 Idempotency and safety

- **Users:** UPSERT ensures re-running sets totals to JSON values rather than duplicating.
- **Sessions:** Designed for one-time import; re-running would re-insert sessions. Avoid re-running or add a guard file if needed.
- **Dry run:** Use `DRY_RUN=1` to validate counts before writing.

6 Step 4: Replace Data Access Layer (Completed)

6.1 Objective

Eliminate runtime reliance on flat files by routing all reads/writes through SQLite while keeping existing route behavior stable.

6.2 What changed

- **SQLite-backed handlers:** `server/data/csvHandler.js` now uses SQLite instead of filesystem CSV. `extttappendToCsv(record)` inserts into `sessions`; `readCsv()` returns rows from `sessions` shaped like CSV records. Existing services (e.g., `statsService`) continue to work unchanged.
- **dbService extensions:** `server/services/dbService.js` now opens the DB in read-write mode and provides new APIs:
`insertSession(record)`, `setScore(pseudo, score)`, `getScore(pseudo)`, `setTime(pseudo, time)`, `getTime(pseudo)`, `computeGlobalAveragePrecision()`.
- **Routes updated:** `server/routes/api.js` no longer writes to `data.json`. Score/time endpoints persist in `users` table; global precision is computed from `sessions`.

6.3 Developer notes

- The legacy `jsonHandler` remains for historical reference but is no longer used by updated routes.
- Precision-related stats are computed on demand from `sessions`; no longer stored redundantly in JSON.
- Existing analytics in `statsService.js` continue to operate via `readCsv()`, now SQLite-backed.

6.4 Implementation details

Files changed

- `server/services/dbService.js`: switched to read-write open; added helpers `setScore`, `getScore`, `setTime`, `getTime`, `computeGlobalAveragePrecision`. Kept existing read APIs.
- `server/data/csvHandler.js`: replaced CSV file I/O with SQLite queries while preserving the same interface (`appendToCsv`, `readCsv`). **Note**: `appendToCsv` now wraps operations in a transaction: insert a session and upsert/increment the user's `sessionCount`.
- `server/routes/api.js`: removed `jsonHandler` usage for scores/times and precision average; routes now call `dbService`.

SQL used Upsert for users (create if absent, increment `sessionCount` otherwise):

```
INSERT INTO users(pseudo, totalScore, totalTime, sessionCount)
VALUES(@pseudo, 0, 0, 1)
ON CONFLICT(pseudo) DO UPDATE SET sessionCount = sessionCount + 1;
```

Set (overwrite) last score/time values for a pseudo:

```
INSERT INTO users(pseudo, totalScore, totalTime, sessionCount)
VALUES(@pseudo, @score, 0, 0)
ON CONFLICT(pseudo) DO UPDATE SET totalScore = excluded.totalScore;
```

```
INSERT INTO users(pseudo, totalScore, totalTime, sessionCount)
VALUES(@pseudo, 0, @time, 0)
ON CONFLICT(pseudo) DO UPDATE SET totalTime = excluded.totalTime;
```

Transaction boundary When recording a session (`/addUser`), we execute within a single transaction to ensure atomicity:

1. Insert into `sessions`.
2. Upsert `users` row and increment `sessionCount` for that pseudo.

Endpoint behavior summary

- `POST /addUser`: inserts a session; auto-creates user row on first session and increments `sessionCount`.
- `POST /saveScore` / `GET /getScore`: store/read score from `users.totalScore`.
- `POST /saveTime` / `GET /getTime`: store/read time from `users.totalTime`.
- `GET /precision_moyenne_globale`: computed on demand from `sessions` (parses Q01 vs. resolution1/2).
- Existing stats endpoints still rely on `statsService + readCsv()`, which now pulls from SQLite.

Why this was straightforward

- `better-sqlite3` is synchronous and simple: it fit the existing code structure without async refactors.
- We preserved the *shape* of CSV access by reimplementing it over SQLite, so analytics code required no changes.
- Upserting into `users` avoided complex branching for first-time vs returning pseudos.

Verification performed

- Required the updated modules and booted the server to confirm no runtime errors.
- Queried DB summary and computed global precision via `dbService` to validate read APIs.
- Inserted a test session for a brand-new pseudo and confirmed: one session row exists and the `users` row is created with `sessionCount=1`.

Troubleshooting

- If Q01/Q02 values carry stray quotes from legacy CSV, precision may compute lower than expected; consider a one-off normalization if needed.
- Ensure `db/database.db` exists (run `npm run db:init`) before exercising the endpoints.

6.5 Data repair: normalize mis-parsed pairs (Completed)

During the CSV $\rightarrow$ SQLite migration, some rows had Q01/Q02 split across Q01..Q04. We added a one-off script to reconstruct proper pairs directly in the database:

- Script: `db/fix-qo-fields.js`
- Logic: load rows, detect pairs without commas, rebuild `Q01=(perceived1,perceived2)` and `Q02=(satisfaction1,satisfaction2)`, then update within a transaction.
- Result: majority of sessions normalized; residual quoting does not affect SQL parsing because queries strip parentheses/quotes.

7 Step 5: Adapt the Service Layer (Completed)

7.1 Objective

Stop reading all sessions in JS and delegate aggregations to SQLite using SQL (`GROUP BY`, conditional `SUM/COUNT`) for performance and correctness.

7.2 What changed

- **Refactor:** `server/services/statsService.js` now opens the DB read-only and issues direct SQL queries instead of iterating arrays from `readCsv()`.
- **Compatibility:** Exported function names kept to avoid breaking routes/UI; some niche functions are stubbed for later SQL ports.
- **Backups:** Originals saved to `backups/step5/` prior to refactor.

7.3 Implementation details

We use CTEs and SQLite string functions to split the stored pairs inline:

```
WITH base AS (
  SELECT resolution1, resolution2,
         TRIM(REPLACE(REPLACE(REPLACE(QO2, '"', ''), ', ', '(' , ')'), ', ', '')) AS
           pair
  FROM sessions
), split AS (
  SELECT resolution1 AS resolution,
         TRIM(SUBSTR(pair, 1, CASE WHEN INSTR(pair, ',') > 0 THEN INSTR(
           pair, ',') - 1 ELSE LENGTH(pair) END)) AS satisfaction
  FROM base
  UNION ALL
  SELECT resolution2,
         TRIM(SUBSTR(pair, CASE WHEN INSTR(pair, ',') > 0 THEN INSTR(pair,
           ',') + 1 ELSE LENGTH(pair) + 1 END))
  FROM base
)
SELECT LOWER(satisfaction) AS satisfaction, resolution, COUNT(*) AS
count
```



```

FROM split
WHERE resolution <> '' AND satisfaction <> ''
GROUP BY resolution, satisfaction;

```

User precision is computed similarly by splitting Q01 into two perceived resolutions and comparing with `resolution1/2`:

```

WITH base AS (
  SELECT resolution1, resolution2,
         TRIM(REPLACE(REPLACE(REPLACE(QO1, '"', ''), '(', ','), ')', '')) AS
           pair
  FROM sessions
  WHERE LOWER(user) = LOWER(@username)
), parts AS (
  SELECT
    resolution1, resolution2, pair,
    TRIM(SUBSTR(pair, 1, CASE WHEN INSTR(pair, ',') > 0 THEN INSTR(pair,
    ',') - 1 ELSE LENGTH(pair) END)) AS p1,
    TRIM(SUBSTR(pair, CASE WHEN INSTR(pair, ',') > 0 THEN INSTR(pair, ',')
    + 1 ELSE LENGTH(pair) + 1 END)) AS p2
  FROM base
), metrics AS (
  SELECT
    SUM(CASE WHEN resolution1 <> '' AND p1 <> '' THEN 1 ELSE 0 END) AS
      total1,
    SUM(CASE WHEN resolution2 <> '' AND p2 <> '' THEN 1 ELSE 0 END) AS
      total2,
    SUM(CASE WHEN resolution1 = p1 THEN 1 ELSE 0 END) AS correct1,
    SUM(CASE WHEN resolution2 = p2 THEN 1 ELSE 0 END) AS correct2
  FROM parts
)
SELECT (total1+total2) AS total, (correct1+correct2) AS correct FROM
  metrics;

```

7.4 Verification

We executed small Node checks to validate shapes and counts of: user precision, global satisfaction, confusions, device and category breakdowns. Outputs matched expectations after the Q0 normalization step.

7.5 Notes

- This change removes full in-memory scans and leverages SQLite for speed and simplicity.
- Minor residual quoting in data is neutralized by SQL-side `REPLACE/TRIM`.
- Remaining detailed functions can be ported to SQL using the same pattern if the UI requires them.

8 Change Log

Step 1

Added dependency `better-sqlite3`. (No code paths yet updated.)

Step 2

Created schema initialization script; introduced `sessions` and `users` tables; added npm script `db:init`.

Step 3

Migrated historical data with `db/migrate-data.js`; added npm script `migrate:data`. Verified counts match sources.

Step 4

Replaced runtime CSV/JSON access with SQLite-backed services. Endpoints now read and write directly to the DB.

Step 5

Refactored stats service to SQL-based aggregations; added one-off data repair for Q01/Q02 pairs.

9 How to Reproduce Current DB State

Run:

```
npm install
npm run db:init
```

This will (re)create the database with the current schema.

Populate from existing CSV/JSON (optional) Run the one-time migration to import current `data.csv` and `data.json`:

```
# Dry run (no writes):
DRY_RUN=1 npm run migrate:data
```

```
# Execute migration:
npm run migrate:data
```

Afterwards, the database contains the historical sessions and users aggregates.

This document will be updated after each subsequent migration task.